

Screen Recording Manuscript

Questions 1 and 2

Martin Davidsen

September 2026

How to use this manuscript

This manuscript is a speaking guide for a screen recording of the assignment notebook. The recording covers only Question 1 and Question 2. Open the notebook in VS Code, select the `bds` kernel, and run the cells from top to bottom. The text below gives the technical explanation, a plain-language explanation, and a suggested way to describe the output.

Do not read every line word for word. Use the headings as landmarks while showing the code and results on screen. The most important point is to explain what each table represents and why each decision was made.

1 Opening the recording

Suggested wording

“In this recording I will walk through the first two questions in the travelling-tracks assignment. I will begin with the setup, then explain how we create one row per track in Question 1. After that I will show how we attach family labels safely in Question 2. I will explain both what the code does and what the results mean.”

Technical explanation

The notebook uses pandas for tables, NumPy for numerical work, and Matplotlib for later visualisation. Questions 1 and 2 only need the data-loading and table operations. The notebook loads two files: the Spotify song associations and the subgenre-to-family lookup table.

Plain-language explanation

The notebook works with two lists of information. The first says which tracks appeared in which playlists. The second is a translation list that tries to place smaller music labels into broader families. Before analysing anything, we need to load both lists and check that they arrived.

2 Setup cell

Code to show

The setup cell imports the tools and loads the data:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd

songs_raw = load_class_csv("spotify_songs.csv", SONGS_URL)
families_raw = load_class_csv("subgenre_families.csv", FAMILIES_URL)

print(f"songs_raw: {songs_raw.shape[0]:,} rows x {songs_raw.shape[1]} columns")
print(f"families_raw: {families_raw.shape[0]:,} rows x {families_raw.shape[1]} columns")
```

Technical explanation

Path allows the loader to look for a local copy of a file. pandas reads the CSV files into DataFrames. A DataFrame is a table with rows and named columns. The loader checks expected local folders first and downloads from the course URL if no local file is found. The final print statements report the dimensions of each table.

Plain-language explanation

This cell prepares the notebook and brings in the two datasets. It first looks on the computer, and if the files are not there, it uses the online course copy. The printed sizes are a quick check that the expected data have been loaded.

Verified result

The setup produced:

- `songs_raw`: 32,833 rows and 23 columns
- `families_raw`: 24 rows and 2 columns

Suggested wording

“The song table has 32,833 rows. These are not necessarily 32,833 different tracks, because the same track can appear in several playlists. The family table has 24 rows and will be used later as a lookup table. The next step is to decide exactly what one row means.”

3 Question 1: What is one row, and how far does each track travel?

3.1 The central decision

Technical explanation

A raw row represents an association between a track and a playlist. It includes playlist information, genre information, and track-level information. Because a track may appear in multiple playlists, raw row counts are not the same as track counts.

For a track-level question, the working table should contain one row per unique `track_id`. We create this table with `groupby` and `agg`:

```
track_summary = (songs_raw.groupby("track_id", as_index=False)
                  .agg(track_name=("track_name", "first"),
                      track_popularity=("track_popularity", "first"),
                      release_date=("track_album_release_date", "first"),
                      playlist_count=("playlist_id", "nunique")))
```

Plain-language explanation

The original table describes appearances in playlists. That is useful when studying playlist associations, but it would give too much weight to a track that appears many times. We therefore make a smaller table where each track appears once. We keep a representative name, popularity score, and release date, and count how many different playlists contain the track.

3.2 Showing one example

Code to show

The notebook selects one track and displays its raw appearances:

```
example_track_id = songs_raw["track_id"].iloc[0]
raw_example = songs_raw.loc[
    songs_raw["track_id"] == example_track_id,
    ["track_id", "playlist_id", "playlist_name",
     "playlist_genre", "playlist_subgenre"]
].drop_duplicates()
display(raw_example)

display(track_summary.loc[
    track_summary["track_id"] == example_track_id
])
```

Technical explanation

`.iloc[0]` selects the first track ID by position. The `.loc` expression filters the raw table to that track and selects the columns needed to explain its playlist appearances. `drop_duplicates()` prevents an identical association from being displayed more than once. The second display looks up the corresponding one-row summary.

Plain-language explanation

First we look at the original evidence. Then we look at the new summary row for the same track. Showing both tables makes the change visible: several playlist appearances become one track row with a playlist count.

Verified result

The example track appeared in two different playlists. The summary table stored one row for it and reported a `playlist_count` of 2.

3.3 Checking that the summary is correct

Code to show

The notebook checks the most important assumption:

```
assert track_summary["track_id"].is_unique
expected_count = songs_raw.loc[
    songs_raw["track_id"] == example_track_id, "playlist_id"
].nunique()
actual_count = int(track_summary.loc[
    track_summary["track_id"] == example_track_id,
    "playlist_count"
].iloc[0])
```

```
assert actual_count == expected_count

print(f"Raw rows for this track: {len(raw_example)}")
print(f"Distinct playlists counted: {actual_count}")
```

Technical explanation

The first assertion checks that `track_id` is unique in the summary. The two counts are calculated independently: one directly from the raw data and one from the summary table. If the values differ, the assertion stops the notebook and reveals a problem.

Plain-language explanation

These checks are like quality controls. They make sure that the summary really has one row per track and that the playlist count was not accidentally changed while building the table.

Verified result

The notebook reported:

```
Raw rows for this track: 2
Distinct playlists counted: 2
Unique tracks: 28,356
Summary rows: 28,356
```

3.4 Release year and missing information

Code to show

The notebook keeps only the year that is actually present in the source:

```
track_summary["release_year"] = pd.to_numeric(
    track_summary["release_date"].astype("string").str[:4],
    errors="coerce")
```

It also checks missing values and date lengths:

```
missing_values = songs_raw[used_columns].isna().sum()
display(missing_values[missing_values > 0])

songs_raw["track_album_release_date"].astype("string").str.len()
```

Technical explanation

The source contains dates with different levels of detail: year only, year and month, or a complete date. Taking the first four characters extracts the year without inventing a month or day. `errors="coerce"` turns an unreadable value into a missing numeric value instead of producing a false year.

Plain-language explanation

Some dates are precise and some are not. If the source only says “2018”, we should not pretend that the track was released on January 1. We keep the honest information we have: the year.

Verified result

The notebook found:

- 5 missing track names
- 1,855 date values with 4 characters, such as a year
- 31 date values with 7 characters, such as year and month
- 30,947 date values with 10 characters, such as a full date
- 0 missing or unparseable release years after extracting the year prefix

3.5 Question 1 conclusion

Suggested wording

“Question 1 showed that a raw row represents a track appearing in a playlist. Since one track can appear several times, I created `track_summary` with one row per unique track and counted distinct playlists. The dataset contains 32,833 raw rows but 28,356 unique tracks. The example track appeared in two playlists, and the checks confirmed that the summary count was correct. I also kept release dates at their observed precision by extracting only the year. This avoids inventing information that was not present in the source.”

4 Question 2: Attach the family label

4.1 Why the lookup must be checked first

Technical explanation

The family table is intended to map each subgenre to one broad family. Before merging, we count how many different family values belong to each subgenre:

```
family_key_counts = families_raw.groupby("subgenre")["family"].nunique()
ambiguous_subgenres = set(
    family_key_counts[family_key_counts > 1].index
)
family_one_to_one = families_raw[
    ~families_raw["subgenre"].isin(ambiguous_subgenres)
].copy()
```

Plain-language explanation

Before joining two tables, we need to check whether the translation list is clear. If one subgenre points to two different families, we cannot safely choose one. Guessing could create duplicate rows or assign the wrong family.

Verified result

The subgenre `dance pop` maps to two families, so it is ambiguous. The song data also contain `neo soul` and `tropical`, which do not appear in the family lookup.

4.2 Predicting the row count

Code to show

The notebook creates unique track-subgenre pairings and counts only those with a clear lookup key:

```
track_subgenres = songs_raw[
    ["track_id", "playlist_subgenre"]
].drop_duplicates()
track_subgenres = track_subgenres.rename(
    columns={"playlist_subgenre": "subgenre"}
)
eligible_associations = track_subgenres[
    track_subgenres["subgenre"].isin(
        family_one_to_one["subgenre"]
    )
].copy()
expected_rows = len(eligible_associations)
print(f"Predicted rows after a one-to-one inner merge: {expected_rows:,}")
```

Technical explanation

The duplicate removal creates one row per observed track-subgenre association. The rename gives both tables the same join-column name. The membership test keeps subgenres present in the checked one-to-one lookup. The resulting length is the expected number of valid rows before the merge.

Plain-language explanation

We first remove repeated copies of the same track and label. Then we count the rows that have a clear family waiting for them. This gives us a prediction to compare with the actual merge result.

Verified result

The predicted number of valid associations was 28,610.

4.3 Performing and checking the merge

Code to show

The merge keeps unmatched rows visible:

```
family_merge = track_subgenres.merge(
    family_one_to_one,
    on="subgenre",
    how="left",
    validate="many_to_one",
    indicator=True
)
display(family_merge["_merge"].value_counts())
```

Technical explanation

A left merge keeps every track–subgenre association from the left table. `validate="many_to_one"` checks that each subgenre matches no more than one family in the cleaned lookup. `indicator=True` adds a column showing whether each row matched.

Plain-language explanation

We want to see both successful and unsuccessful matches. The merge does not hide the problem cases. It labels each row as matched or unmatched, and it stops if the supposed one-to-one lookup is not actually safe.

Verified result

The merge produced:

```
both 28610
left_only 4223
right_only 0
```

This means 28,610 associations received a family label and 4,223 did not.

4.4 Keeping only defensible matches

Code to show

The notebook keeps the valid matches and records the excluded associations:

```
family_associations = family_merge.loc[
    family_merge["_merge"] == "both",
    ["track_id", "subgenre", "family"]
].copy()
unmatched_associations = family_merge.loc[
    family_merge["_merge"] == "left_only"
].copy()

assert len(family_associations) == expected_rows
assert family_associations["subgenre"].isin(
    ambiguous_subgenres
).sum() == 0
```

Technical explanation

The `both` filter keeps only rows where a track association and a family value were found. The assertions confirm that the final valid count equals the prediction and that no ambiguous subgenre remains in the accepted data.

Plain-language explanation

We keep the rows we can explain and set aside the rows we cannot classify safely. This costs us some data, but it is better than making up a family label or counting an ambiguous row twice.

Verified result

The notebook reported:

```
Valid family associations kept: 28,610
Associations without a family label: 4,223
Decision: exclude ambiguous and unmapped associations rather than invent a family
label.
```

4.5 Question 2 conclusion

Suggested wording

“Question 2 showed why a lookup table should be checked before it is joined to the data. The subgenre dance pop had two possible family labels, while neo soul and tropical were missing from the lookup. I therefore removed the ambiguous mapping before the merge and kept only one-to-one matches. I predicted 28,610 valid associations, and the merge produced exactly that number. A further 4,223 associations had no family label. The cost is that these rows cannot be used in family-level comparisons, but the remaining results are easier to defend.”

5 Closing the recording

Suggested wording

“The main lesson from these two questions is that data analysis starts before the final calculation. First, I defined what a row meant and created a track-level table. Then I checked the family lookup before joining it to the track associations. The code uses assertions and merge checks to make hidden mistakes visible. Questions 1 and 2 are now complete. Question 3 is left for a later recording.”

Final recording checklist

- Select the `bds` kernel in VS Code.
- Start at the setup cell and run the notebook in order.
- Show the raw example and the one-row track summary in Question 1.
- Point out the assertions and explain why they matter.
- Show the predicted row count before running the Question 2 merge.
- Explain `both` and `left_only` in the merge output.
- State the decision to exclude ambiguous and unmapped family labels.
- End after Question 2; do not present Question 3 in this recording.